

Unidad 03

Estudio de la estructura de la comunidad

2025

Guía de práctica

Facultad de Ciencias Exactas y Naturales

Universidad Nacional del Nordeste

1 Introducción

Aquí veremos distintas formas de medir y representar la estructura de una comunidad. Se utilizarán los datasets `abundancia.csv`, `ambiente.csv`, `abundancia_conservacion.csv` y `abundancia_conservacion_larga.csv`.

NOTA: Es recomendable que trabajen dentro de un proyecto de RStudio.

2 Carga de librerías y datos

```
library(tidyverse)
library(BiodiversityR)
library(betapart)
library(ggplot2)
library(ggrepel)
library(ggforce)
library(concaveman)
library(iNEXT)
library(factoextra)

set.seed(2025)

# Para diversidad beta
abund_conservacion <- read.csv("datos/abundancia_conservacion.csv", header =
  TRUE, row.names = 1)

# Para curvas de Whittaker y acumulación, y análisis NMDS
abundancia <- read.csv("datos/abundancia.csv", header = TRUE, row.names = 1)
ambiente <- read.csv("datos/ambiente.csv", header = TRUE, row.names = 1)

# Para curvas de rarefacción
abund_larga <- read.csv("datos/abundancia_conservacion_larga.csv", header =
  TRUE, row.names = 1)
```

3 Diversidad Beta

Para los calculos de diversidad beta es necesario datos de incidencia, para eso transformaremos nuestra matriz de abundancia a una de incidencia con la funcion `decostand()`. Y crearemos el objeto principal (*core object*) que usaremos en los calculos usando `betapart.core()`.

```
# Transformación de abundancia a incidencia (pa = presence absence)
incidencia <- decostand(abund_conservacion, method = "pa")

div_beta <- betapart.core(incidencia)
```

Dismilitud entre multiples sitios

Calcula el recambio (*turnover*) y anidamiento (*nestedness*) de especies entre todos los sitios.

```
## En base Jaccard
multi_jac <- beta.multi(div_beta, index.family = "jac")
multi_jac

## En base Sørensen
multi_sor <- beta.multi(div_beta, index.family = "sor")
multi_sor

## Gráficos

# Jaccard
multi_jac_df <- data.frame(
  Componente = c("Recambio ( $\beta_{jt}$ )", "Anidamiento ( $\beta_{jne}$ )", "Total ( $\beta_{jac}$ )"),
  Valor = unlist(multi_jac)
)

ggplot(multi_jac_df, aes(x = Componente, y = Valor, fill = Componente)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = round(Valor, 2)), vjust = -0.5) +
  scale_fill_brewer(palette = "Set2") +
  labs(y = "Disimilitud", x = "Componente") +
  ylim(0, 1) +
  theme_classic()
```

```
# Sorensen
multi_sor_df <- data.frame(
  Componente = c("Recambio ( $\beta_{sim}$ ", "Anidamiento ( $\beta_{sne}$ ", "Total ( $\beta_{sor}$ )"),
  Valor = unlist(multi_sor)
)

ggplot(multi_sor_df, aes(x = Componente, y = Valor, fill = Componente)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = round(Valor, 2)), vjust = -0.5) +
  scale_fill_brewer(palette = "Set2") +
  labs(y = "Disimilitud", x = "Componente") +
  ylim(0, 1) +
  theme_classic()
```

Disimilitud entre pares

Calcula la disimilitud entre pares de sitios, generando una matriz de distancias. Para visualizar estas distancias realizaremos un gráfico de cluster.

```
# Base Jaccard
pair_jac <- beta.pair(div_beta, index.family = "jac")
pair_jac

# Base Sorensen
pair_sor <- beta.pair(div_beta, index.family = "sor")
pair_sor

## Gráficos

# Se crean los objetos hclust
hclust_jac <- hcut(pair_jac$beta.jac, hc_method = "average", hc_func =
  "hclust")
hclust_sor <- hcut(pair_sor$beta.sor, hc_method = "average", hc_func =
  "hclust")

# Cluster Jaccard
fviz_dend(hclust_jac, palette = "Set2", horiz = TRUE, main = "Dendograma
  Jaccard")
```

```
# Cluster Sorensen
fviz_dend(hclust_sor, palette = "Set2", horiz = TRUE, main = "Dendograma
  Sorensen")

# Gráficos de barras apiladas
#
# Transformamos los datos en formato largo para ggplot

pairs_pivot <- function(datos, nombre_columna) {
  as.matrix(datos) %>%
    as.data.frame() %>%
    rownames_to_column("grupo1") %>%
    pivot_longer(
      !grupo1,
      names_to = "grupo2",
      values_to = nombre_columna
    ) %>%
    filter(grupo1 < grupo2) %>%
    mutate(sitios = paste(grupo1, grupo2, sep = "-")) %>%
    select(sitios, nombre_columna)
}

## Base Jaccard
beta_jtu <- pairs_pivot(pair_jac$beta.jtu, "βjtu")
beta_jne <- pairs_pivot(pair_jac$beta.jne, "βjne")

# Unimos los componentes en una sola tabla
pair_jac_long <- left_join(beta_jtu, beta_jne, by = "sitios") %>%
  pivot_longer(!sitios, names_to = "Componente", values_to = "Valor") %>%
  mutate(Componente = factor(Componente, levels = c("βjne", "βjtu")))
pair_jac_long

## Base Sorensen
beta_sim <- pairs_long(pair_sor$beta.sim, "βsim")
beta_sne <- pairs_long(pair_sor$beta.sne, "βsne")

# Unimos los componentes en una sola tabla
```

```

pair_sor_long <- left_join(beta_sne, beta_sim, by = "sitios") %>%
  pivot_longer(!sitios, names_to = "Componente", values_to = "Valor") %>%
  mutate(Componente = factor(Componente, levels = c("βsne", "βsim")))
pair_sor_long

# Gráfico Jaccard
plot_pairs_jac <- ggplot(pair_jac_long, aes(x = sitios, y = Valor, fill =
  Componente)) +
  geom_bar(stat = "identity", color = "black", width = 0.6) +
  scale_fill_brewer(palette = "Set3") +
  geom_label(
    aes(label = round(Valor, 2)),
    position = position_stack(vjust = 0.5),
    size = 3,
    color = "grey20",
    show.legend = FALSE,
  ) +
  labs(
    x = "",
    y = expression(beta["JAC"]),
    fill = "Componente",
    title = "Partición de la diversidad beta\n(índice de Jaccard)"
  ) +
  theme_classic(base_size = 13) +
  theme(
    legend.position = "bottom",
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
    axis.text = element_text(color = "black")
  )
plot_pairs_jac

# Gráfico Sorensen
plot_pairs_sor <- ggplot(pair_sor_long, aes(x = sitios, y = Valor, fill =
  Componente)) +
  geom_bar(stat = "identity", color = "black", width = 0.6) +
  scale_fill_brewer(palette = "Set3", breaks = c("βsim", "βsne")) +
  geom_label(

```

```

aes(label = round(Valor, 2)),
position = position_stack(vjust = 0.5),
size = 3,
color = "grey20",
show.legend = FALSE,
) +
labs(
  x = "",
  y = expression(beta["SOR"]),
  fill = "Componente",
  title = "Partición de la diversidad beta\n(índice de Sorensen)"
) +
theme_classic(base_size = 13) +
theme(
  legend.position = "bottom",
  panel.grid.major.x = element_blank(),
  panel.grid.minor = element_blank(),
  axis.text = element_text(color = "black")
)
plot_pairs_sor

```

Estos gráficos están basados en ggplot, así que es posible personalizarlos como cualquier otro gráfico de ggplot.

4 Curva de Whitaker

```

# Variable categórica como factor
ambiente$estado_conservacion <- factor(ambiente$estado_conservacion, levels
  = c("ECB", "ECI", "ECD"))

rank_abundancia <- rankabuncomp(
  abundancia,
  y = ambiente,
  factor = "estado_conservacion",
  legend = FALSE
)

```

```

# Funcion para graficar grupos de forma independiente
curva_whitt <- function(datos, grupo, color) {
  x <- subset(datos, Grouping == grupo)
  plot <- ggplot(x, aes(x = rank, y = abundance)) +
    geom_line(color = color) +
    geom_point(size = 2.5, color = color) +
    # Muestra los nombres de las primeras tres especies
    geom_text_repel(
      data = subset(datos, labelit == TRUE & Grouping == grupo),
      aes(label = species),
      force_pull = -0.01
    ) +
    labs(x = "Rank", y = "Abundancia") +
    theme_classic()
  plot
}

curva_whitt(rank_abundancia, "ECB", "darkgreen")
curva_whitt(rank_abundancia, "ECI", "orange")
curva_whitt(rank_abundancia, "ECD", "red")

# Gráfico con todos los grupos
ggplot(rank_abundancia, aes(x = rank, y = abundance, color = Grouping)) +
  geom_line() +
  geom_point(size = 2.5) +
  scale_color_manual(breaks = c("ECB", "ECI", "ECD"), values =
    c("darkgreen", "orange", "red")) +
  labs(x = "Rank", y = "Abundancia", color = NULL, shape = NULL) +
  theme_classic() +
  theme(legend.position = "top")

```

5 Curvas de acumulación

```

curva <- specaccum(abundancia)
curva

```



```
# Tabla para ggplot
datos_sp <- data.frame(
  Sitios = curva$sites,
  Riqueza = curva$richness,
  SD = curva$sd
)

# Grafico
ggplot(datos_sp, aes(x = Sitios, y = Riqueza)) +
  geom_ribbon(aes(ymin = Riqueza - SD, ymax = Riqueza + SD), fill =
    "grey90") +
  scale_x_continuous(breaks = datos_sp$Sitios) +
  geom_line(color = "blue") +
  theme_classic()
```

6 Curvas de rarefacción

```
rarefaccion <- iNEXT(abund_larga, q = c(0, 1, 2), datatype = "abundance")
rarefaccion
```

El argumento `q` nos permite elegir para que valores de `q` queremos calcular la diversidad, puede tomar cualquier valor/es. Y `datatype` nos permite elegir con que tipo de datos estamos realizando los cálculos, en este caso tenemos datos de abundancia, pero también es posible usar datos de incidencia.

Gráficos

```
# Gráficos por tratamiento
plot_ec <- ggiNEXT(rarefaccion, type = 1, facet.var = "Assemblage") +
  theme_classic(base_size = 10) +
  theme(legend.position = "bottom")
plot_ec
```

```
# Gráficos por orden q
plot_orderq <- ggiNEXT(rarefaccion, type = 1, facet.var = "Order.q") +
  theme_classic(base_size = 10) +
  theme(legend.position = "bottom")
plot_orderq
```

7 NMDS (Escalado multi-dimensional no métrico)

```
resultado_nmds <- metaMDS(abundancia, distance = "bray", K = 2)
resultado_nmds

# Stress
resultado_nmds$stress

# ANOSIM
dist_sitios <- vegdist(abundancia)
anosim_sitios <- anosim(dist_sitios, ambiente$estado_conservacion, distance
  = "bray")
summary(anosim_sitios)
```

Gráficos

```
# Creamos un data.frame con los resultados
puntos_nmds <- as.data.frame(resultado_nmds$points)
puntos_nmds$CONSERVACION <- ambiente$estado_conservacion

# Agregamos siglas para nombres de los sitios y guardamos el valor de stress
puntos_nmds$SITIO <- abbreviate(rownames(abundancia))
estres <- paste("Stress =", round(resultado_nmds$stress, 2))

# Graficamos
plot_nmds <- ggplot(puntos_nmds, aes(x = MDS1, y = MDS2)) +
  ggtitle("NMDS") +
  geom_point(aes(shape = CONSERVACION), size = 3) +
```

```
scale_shape_manual(  
  name = "",  
  breaks = c("ECB", "ECI", "ECD"),  
  labels = c("Conservado", "Intermedio", "Degradado"),  
  values = c(15, 16, 17)  
) +  
geom_mark_hull(  
  aes(group = CONSERVACION, linetype = CONSERVACION),  
  concavity = 10,  
  radius = 0,  
  expand = 0,  
  show.legend = FALSE  
) +  
scale_linetype_manual(values = c("solid", "dashed", "dotted")) +  
annotate("text", x = +Inf, y = +Inf, label = estres, hjust = 1, vjust = 1)  
+  
theme_classic()  
plot_nmds  
  
# Opcionalmente podemos añadir el nombre de los sitios  
plot_nmds +  
  geom_text_repel(  
    aes(label = SITIO),  
    box.padding = 0.5,  
    size = 3.5,  
    colour = "blue",  
  )
```